

REFACTORY ACADEMY · APPLICATION SECURITY MODULE

# Course Overview

JAVASCRIPT / NODE + PYTHON /  
DJANGO TRACKS

OAS Bay Project · 5 Classes · 2 Hours Each · Beginner Level

## The 5 Days at a Glance

| Day   | Focus                                                                                                     | OWASP Categories      |
|-------|-----------------------------------------------------------------------------------------------------------|-----------------------|
| Day 1 | Introduction to Application Security: the CIA Triad, the OWASP Top 10, and why security is everyone's job | Overview of A01 - A10 |
| Day 2 | Injection Attacks: SQL Injection and input validation                                                     | A03, Injection        |
| Day 3 | Cross-Site Scripting (XSS): stored, reflected, and how to render user data safely                         | A03, Injection (XSS)  |
| Day 4 | Authentication & Cryptographic Failures: password hashing, sessions, MFA                                  | A07 + A02             |
| Day 5 | Access Control, Secure Design & Wrap-Up: IDOR, CSRF, misconfiguration, final review                       | A01, A04, A05 + recap |

## DAY 1

# Introduction to Application Security

*The CIA Triad, the OWASP Top 10, and why security is everyone's job*

| 0:00 - 0:25<br>Theory                | 0:25 - 0:55<br>Activity                | 0:55 - 1:35<br>Practical      | 1:35 - 2:00<br>Review             |
|--------------------------------------|----------------------------------------|-------------------------------|-----------------------------------|
| <i>CIA Triad &amp; Why Security?</i> | <i>OWASP Top 10<br/>+ Case Studies</i> | <i>Spot the Vulnerability</i> | <i>Discussion + Day 2 Preview</i> |

## Learning Objectives

- Explain the CIA Triad in plain language, with one OAS Bay example of each
- List and briefly describe all 10 OWASP Top 10 vulnerabilities
- Spot at least 3 vulnerabilities in a short code snippet
- Explain why security is a mindset carried through the whole build, not a final step

## OAS Bay Tie-In

Every example uses OAS Bay's booking, parts, and payment flows, so students immediately see security concepts inside a business they already understand: a car service bay, not an abstract app.

## Track Focus

- JavaScript/Node.js: hardcoded client-side checks, innerHTML risks, and localStorage misuse in a Node.js backend
- Python/Django: the same vulnerability classes shown in Django views and templates, including the |safe filter

## Homework Before Next Class

- Read the OWASP overview for A03:2021 Injection (15 minutes)
- Think about what happens if a user types ' OR '1'='1 into the OAS Bay login form
- Optional: read 3 more OWASP Top 10 descriptions on [owasp.org](https://owasp.org)

## DAY 2

# Injection Attacks

*SQL Injection and input validation, using the OAS Bay parts database*

| 0:00 - 0:25<br>Theory                                                | 0:25 - 0:55<br>Activity                                              | 0:55 - 1:35<br>Practical                                                 | 1:35 - 2:00<br>Review             |
|----------------------------------------------------------------------|----------------------------------------------------------------------|--------------------------------------------------------------------------|-----------------------------------|
| <i>What injection is, and why string-built queries are dangerous</i> | <i>Live SQL injection demo against a local OAS Bay test database</i> | <i>Rewrite vulnerable queries using parameterised queries and an ORM</i> | <i>Discussion + Day 3 Preview</i> |

## Learning Objectives

- Explain how SQL Injection works, step by step
- Identify SQL Injection in both raw SQL and ORM-adjacent code
- Fix an injectable query using parameterised queries
- Use an ORM (Sequelize or Django ORM) correctly and explain why it protects against injection by default

## OAS Bay Tie-In

Students attack, then fix, the OAS Bay parts search and supplier credit forms: the exact features introduced on Day 1.

## Track Focus

- JavaScript/Node.js: parameterised queries with a Node.js SQL driver, and safe use of Sequelize
- Python/Django: raw cursor.execute risks versus safe Django ORM filtering (`Parts.objects.filter(...)`)

## Homework Before Next Class

- Read the OWASP overview for A03:2021, XSS section (10 minutes)
- Try to explain, in your own words, the difference between stored and reflected XSS

## DAY 3

# Cross-Site Scripting (XSS)

*Stored and reflected XSS, and how to render user data safely*

| 0:00 - 0:25<br>Theory                                       | 0:25 - 0:55<br>Activity                                    | 0:55 - 1:35<br>Practical                                  | 1:35 - 2:00<br>Review             |
|-------------------------------------------------------------|------------------------------------------------------------|-----------------------------------------------------------|-----------------------------------|
| <i>How XSS works, and why it targets users, not servers</i> | <i>Case study: the British Airways payment page breach</i> | <i>Break, then fix, an OAS Bay supplier-notes feature</i> | <i>Discussion + Day 4 Preview</i> |

## Learning Objectives

- Distinguish stored XSS from reflected XSS
- Explain why innerHTML (JavaScript) and the |safe filter (Django templates) are dangerous with user data
- Apply textContent and Django's default autoescaping correctly
- Describe what a Content Security Policy (CSP) header does

## OAS Bay Tie-In

Students inject a script into OAS Bay's supplier notes field, watch it fire, then patch the same feature safely.

## Track Focus

- JavaScript/Node.js: innerHTML vs textContent, and a basic CSP header on an Express response
- Python/Django: Django's default autoescaping, and when the |safe filter becomes a real risk

## Homework Before Next Class

- Read about password hashing (bcrypt or argon2), just the concept, 10 minutes
- Think about why a session should expire after a period of inactivity

## DAY 4

# Authentication & Cryptographic Failures

*Password hashing, session management, and multi-factor authentication*

| 0:00 - 0:25<br>Theory                                                        | 0:25 - 0:55<br>Activity                             | 0:55 - 1:35<br>Practical                                                       | 1:35 - 2:00<br>Review             |
|------------------------------------------------------------------------------|-----------------------------------------------------|--------------------------------------------------------------------------------|-----------------------------------|
| <i>How authentication fails in practice, and what strong auth looks like</i> | <i>Case study: the Facebook 'View As' token bug</i> | <i>Add password hashing and session expiry to the OAS Bay technician login</i> | <i>Discussion + Day 5 Preview</i> |

## Learning Objectives

- Explain why plain-text and reversible password storage is never acceptable
- Use bcrypt/argon2 (Node.js) or Django's built-in password hashers correctly
- Configure session expiry and secure cookie flags (HttpOnly, Secure, SameSite)
- Describe what MFA adds, even when a password is already known to an attacker

## OAS Bay Tie-In

Students secure the OAS Bay technician and admin login flows, and protect stored customer payment information end to end.

## Track Focus

- JavaScript/Node.js: bcrypt hashing and cookie flags in a Node.js/Express session
- Python/Django: Django's built-in authentication system and password hashers, session settings in settings.py

## Homework Before Next Class

- Read the OWASP overview for A01:2021 Broken Access Control (10 minutes)
- Think of one OAS Bay feature where hiding a button in the UI would NOT be enough to stop misuse

## DAY 5

# Access Control, Secure Design & Wrap-Up

*IDOR, CSRF, security misconfiguration, and a full OWASP Top 10 recap*

| 0:00 - 0:25<br>Theory                                                                  | 0:25 - 0:55<br>Activity                         | 0:55 - 1:35<br>Practical                                                    | 1:35 - 2:00<br>Review                                |
|----------------------------------------------------------------------------------------|-------------------------------------------------|-----------------------------------------------------------------------------|------------------------------------------------------|
| <i>Broken access control, IDOR, and CSRF, tied back to the STRIDE model from Day 1</i> | <i>Full OWASP Top 10 recap quiz, as a class</i> | <i>Mini security audit of the complete OAS Bay booking system, in pairs</i> | <i>Module wrap-up, feedback, and what comes next</i> |

## Learning Objectives

- Explain Broken Access Control and IDOR with an OAS Bay example of each
- Explain what CSRF is and how SameSite cookies and CSRF tokens defend against it
- Run a basic STRIDE threat model on a new feature unassisted
- Complete a checklist-based security audit of a small system

## OAS Bay Tie-In

In pairs, students audit a finished OAS Bay module (booking, payments, or parts) against the full OWASP Top 10 checklist and present one finding to the class.

## Track Focus

- JavaScript/Node.js: Express route-level role checks and CSRF middleware
- Python/Django: Django's built-in CSRF protection and permission/role checks in views

## Homework Before Next Class

- No formal homework: review your Day 1 to Day 5 notes ahead of the final project security review

## Where This Fits in Your Final Project

---

The security habits built across these 5 days apply directly to your OAS Bay records management system: authentication for technicians and admins, safe handling of customer and payment data, input validation on every form, and access control on every route. Treat this module as the security checklist for your own build, not a separate topic.

*Refactory Academy - Application Security Module*  
*5 Classes | JavaScript/Node & Python/Django Tracks | OAS Bay Project*